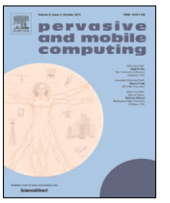
Pervasive and Mobile Computing 88 (2023) 101738



Contents lists available at ScienceDirect

Pervasive and Mobile Computing

journal homepage: www.elsevier.com/locate/pmc



PPSS: A privacy-preserving secure framework using blockchain-enabled federated deep learning for Industrial IoTs



Djallel Hamouda [a], Mohamed Amine Ferrag [b,*], Nadjette Benhamida [a], Hamid Seridi [a]

[a] Labstic Laboratory, Department of Computer Science, Guelma University, B.P. 401, 24000, Algeria
[b] Technology Innovation Institute, 9639 Masdar City, Abu Dhabi, United Arab Emirates

ARTICLE INFO

Article history:
Received 18 June 2022
Received in revised form 12 December 2022
Accepted 20 December 2022
Available online 23 December 2022

Keywords:
Federated learning
Intrusion detection
Privacy preserving
Proof of learning
Security
Industrial internet of things (IIoT)

ABSTRACT

The growing reliance of industry 4.0/5.0 on emergent technologies has dramatically increased the scope of cyber threats and data privacy issues. Recently, federated learning (FL) based intrusion detection systems (IDS) promote the detection of large-scale cyber-attacks in resource-constrained and heterogeneous industrial systems without exposing data to privacy issues. However, the inherent characteristics of the latter have led to problems such as a trusted validation and consensus of the federation, unreliability, and privacy protection of model upload. To address these challenges, this paper proposes a novel privacy-preserving secure framework, named PPSS, based on the use of blockchain-enabled FL with improved privacy, verifiability, and transparency. The PPSS framework adopts the permissioned-blockchain system to secure multi-party computation as well as to incentivize cross-silo FL based on a lightweight and energy-efficient consensus protocol named Proof-of-Federated Deep-Learning (PoFDL). Specifically, we design two federated stages for global model aggregation. The first stage uses differentially private training of Stochastic Gradient Descent (DP-SGD) to enforce privacy protection of client updates, while the second stage uses PoFDL protocol to prove and add new model-containing blocks to the blockchain. We study the performance of the proposed PPSS framework using a new cyber security dataset (Edge-IIoT dataset) in terms of detection rate, precision, accuracy, computation, and energy cost. The results demonstrate that the PPSS framework system can detect industrial IIoT attacks with high classification performance under two distribution modes, namely, non-independent and identically distributed (Non-IID) and independent and identically distributed (IID).

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Industrial Internet of Things (IIoT) is a new paradigm that characterizes the smart industry featured with the emergent technologies such as internet of things (IoT), Cyber Psychical Systems (CPS), cloud/fog computing, deep learning, etc, to provide autonomy, reliability and optimize business efficiency and service quality [1]. However, the interconnections of IIoT infrastructure using in-secured design protocols and public networks (internet) to transmit sensitive information to Cloud-enabling applications have raised security and privacy issues, holding back the adoption of these technologies.

* Corresponding author.
E-mail addresses: hamouda.djallel@univ-guelma.dz (D. Hamouda), mohamed.ferrag@tii.ae (M.A. Ferrag), benhamida.nadjette@univ-guelma.dz (N. Benhamida), hamid.seridi@univ.guelma.dz (H. Seridi).

https://doi.org/10.1016/j.pmcj.2022.101738
1574-1192/© 2022 Elsevier B.V. All rights reserved.

Industrial organizations have become an eligible target for cybercriminals conducting many devastating cyber-attacks that have the potential to lead to real-world negative effects. These attacks vary from privacy breaches to exploiting backdoors and disrupting services of various critical resources.

An intrusion detection system (IDS) is a promising cyber security defense mechanism used for protecting industrial networks and control systems by detecting any potential security threats on time. However, developing an effective and efficient IDS in resource-limited environments brings important challenges regarding many issues: industrial devices are resource-constrained and lack the ability of local processing of the majority of ML-based and DL-based IDS detection approaches. Industrial data are insufficient, heterogeneous, and large-scale by nature, data privacy issues present a real challenge and restricts centralized processing offered by cloud services, thus requiring personalized and sophisticated detection methodologies [2]. To overcome these challenges federated learning (FL) was proposed as a decentralized and privacy-preserving computing paradigm that enables both machine-learning-based (ML) and deep-learning-based (DL) detection models to be trained locally on edge devices and only sharing updates for the global model optimization instead of sensitive data [3]. Cross-silo FL allows different industrial organizations to exchange and share intrusion events, incident logs, and reported alert data on cyber attacks with each other through transfer learning and without leaking privacy. However, this federation process still needs a trusted validation server, a consensus mechanism for federation, a secured aggregation of information and coordination between the participating clients. Moreover, model updates may include traces that can be used to infer private and sensitive information, adversarial attacks such as data poisoning and inference attacks still find flaws through audit gateways and form major security threats against the privacy and the reliability of model updates. For that, it is requiring mixing FL with other techniques [4].

The integration of Industrial IoTs systems with blockchain technologies offers substantial privacy and security solutions for decentralized networks, providing validated and real-time data to participants using smart contracts-based verification. Therefore, blockchain can be integrated with FL to serve as an immutable and distributed database for the federated learning system to validate and store model updates, and secure audit gateways using authenticated P2P communication between Edge servers and IoT devices [5]. Based on blockchain technology, the FL models can be securely and frequently updated through distributed training on Edge devices while maintaining data privacy and resource efficiency. However, adopting blockchain with different aspects and settings of federated learning is challenging. Additional computing resources are needed for transaction execution and verification, efficient and valid proof of learning as well as a fair consensus mechanism is required between distrusted participants. The privacy breaches still occur in honest-but-curious scenarios, when participants may negatively exploit received updates.The main motivations of this paper are as follows:

- Designing a privacy-preserving secure scheme for federated learning that can efficiently protect confidential data from disclosure during processing, training, and model sharing in resource-limited Industrial IoTs.
- Improve the overall training process using less centralized FL and more energy-efficient consensus algorithm among multiple organizations and across resource-constrained devices.
- Developing a privacy-preserving IDS while maintaining the system efficiency and effectiveness, to protect from large-scale and emergent cyber attacks.

To address the aforementioned issues, and have industrial organizations consolidate and overcome common security challenges, we propose a novel privacy-preserving secure framework, named PPSS, using blockchain-enabled FL to provide security against large-scale and emergent cyber attacks, with improved privacy, reliability, and transparency. We also adopt the blockchain as an incentivized mechanism for cross-silo FL using cryptocurrency in order to overcome data shortage and model efficiency. The main contributions of this paper are as follows:

- We propose a novel blockchain-based scheme to enable the cross-silo FL and facilitate the exchange of security models through transfer learning among organizations.
- We propose a novel privacy-preserving and secure framework named PPSS that combines differentially private training with an energy-efficient consensus protocol named Proof-of-Federated Deep-Learning (PoFDL) in order to verify, validate and store new efficient models on the blockchain.
- We prove the security of the proposed PPSS framework against byzantine and Sybil attacks, honest-but-curious and common adversarial attacks.
- We evaluate the performance of the proposed PPSS framework using our recently published Edge-IIoTset dataset, and the convolutional neural network (CNN) in different FL settings.

The remainder of this paper is organized as follows. We review related work in Section 2. We overview the design goal of our proposed PPSS framework in Section 3.1. Section 4 demonstrates the deployment architecture and the security elements of PPSS. Section 5 demonstrates experimental results and effectiveness of the proposed PPSS-based IDS compared with recent state-of-the-art security frameworks. Finally, we conclude our work in Section 6.

2. Related works

2.1. Federated learning-based intrusion detection

Recently, federated-learning-based IDSs have attracted high interest in academia, which have been proposed as privacy-preserving schemes for IDS during model learning against various IIoT security requirements and challenges.

Popoola et al. [6] proposed a Federated-DL-based (FDL) method for botnet attack detection to prevent data privacy leakage in IoT edge devices. The proposed method used the federated averaging (FedAvg) algorithm in order to aggregate local model updates after a number of communication rounds. The experiment results showed that the proposed FDL model outperformed all centralized (Cloud-centric), localized (on-device), and Distributed DL methods in terms of classification performance, privacy-preserving, memory, and communication efficiency, but it takes a long training time which can affect the energy cost. Rahman et al. [7] proposed an FL-based privacy-preserving scheme for IoT intrusion detection. Their scheme uses on devices models training then aggregates only resulted updates. The authors concluded that FL-based detection outperforms the self-learning approach due to knowledge sharing and it might obtain equivalent accuracy with respect to the centralized approach that has global insights into the entire system.

Al-Marri at al. [8] presented a federated mimic learning for distributed IDS in order to limit the risk of reverse engineering attacks that may learn information about the user's data from updates. Experimental results demonstrated that their proposed model provides similar detection accuracy to DL models while maintaining privacy. Li et al. [9] proposed a distributed network IDS approach using FL to solve limited resources and high privacy requirements of satellite networks. The results demonstrate that the proposed approach outperformed traditional methods in terms of detection rate and CPU utilization. Li et al. [10] proposed a federated-DL-based IDS system, named DeepFed, for industrial Cyber–Physical Systems (CPS). The DeepFed system uses a convolutional neural network and a gated recurrent unit for privacy-preserving and efficient detection of various security attacks. In addition, the DeepFed system proposed a Paillier cryptosystem-based secure protocol to preserve the model parameters through the training process.

2.2. Blockchain with federated deep learning

To overcome the trust issues over a decentralized network, blockchain-enabling deep learning systems have been also successfully proposed to secure training data from falsification and injection attacks. Kumar et al. [11] proposed a privacy-preserving secure framework using blockchain and DL for intelligent transport systems (ITS). The blockchain was designed to securely transmit the ITS data between vehicles and other units, by employing smart contract-based enhanced Proof of Work (PoW) to verify data integrity and mitigate data poisoning threats. The DL module was designed for encoding data into a new format to prevent inference attacks. Then, the encoded data was used for intrusion detection based on a Recurrent Neural Network (RNN).

Ferrag et al. [12] designed an IDS framework to detect attacks and fraudulent transactions in the blockchain-based energy network. The proposed framework combines the blockchain with DL-based RNN into one architecturally secure framework for the smart grid, where blockchain was used to achieve privacy preservation and DL for attack detection. Kumar et al. [13] proposed an intelligent blockchain framework that integrates blockchain with ML-based IDS to protect privacy and security in IoT. The blockchain module was designed to secure transmitted IoT data from data poisoning attacks. Authenticated data were then encoded using Principal Component Analysis (PCA) to avoid inference attack data before being forwarded to anomaly detection.

Kumar et al. [14] developed a distributed framework to detect DDoS attacks in smart contracts blockchain-IoT systems. The authors suggested an interplanetary file system (IPFS) for off-chain data storage as edge intelligence to address scalability issues and proposed ML-based IDS integrated with smart contract based on fog computing to mitigate DDoS attacks. In [15], the authors proposed a secured Privacy-Preserving Framework, named SP2F, for smart agriculture using blockchain and smart contract for data authentication and to mitigate data poisoning attacks. Then, a sparse AutoEncoder (SAE) was also applied to transform data and prevent inference attacks. In addition, the authors proposed anomaly detection based on the Long-Short-Term Memory (LSTM) DL approach to enhance security against security attacks.

Wan et al. [16] designed a blockchain-enabled decentralized FL to alleviate the data falsification issues while reducing communication costs between cloud and edge. Specifically, the authors proposed a Wasserstein generative adversarial network (WGAN) with differential privacy (DP) to protect the model parameters while improving the data utility of edge devices. Their proposed model was evaluated considering different aspects of convergence efficiency, accuracy, and data utility. In [17], the authors proposed a privacy-encoding-based Federated Learning (PEFL) framework for privacy protection and intrusion detection in smart agriculture. The PEFL framework uses two-level privacy-preserving modules: perturbation-based and DL-based privacy using feature mapping encoding techniques in the former, and Long-Short Term Memory-AutoEncoder (LSTM-AE) for data transformation in the latter, in order to protect data from inference attacks during the FL process. Then, the PEFL framework uses an FL-based Gated Recurrent Unit (FedGRU) for intrusion detection, deployed using cloud and edge devices. However, it is still challenging to adopt blockchain with several FL aspects, especially in the context of IIoTs. In addition, how to design a robust and efficient FL-based IDS system on top of a blockchain has not been widely discussed.

3. Design goal & preliminaries

3.1. Design goal

To overcome the common industrial IoTs issues such as data availability and computation resources restrictions in most ML and DL based industrial applications (i.e., faults detection, intrusion detection, etc.), We aim to expand the FL

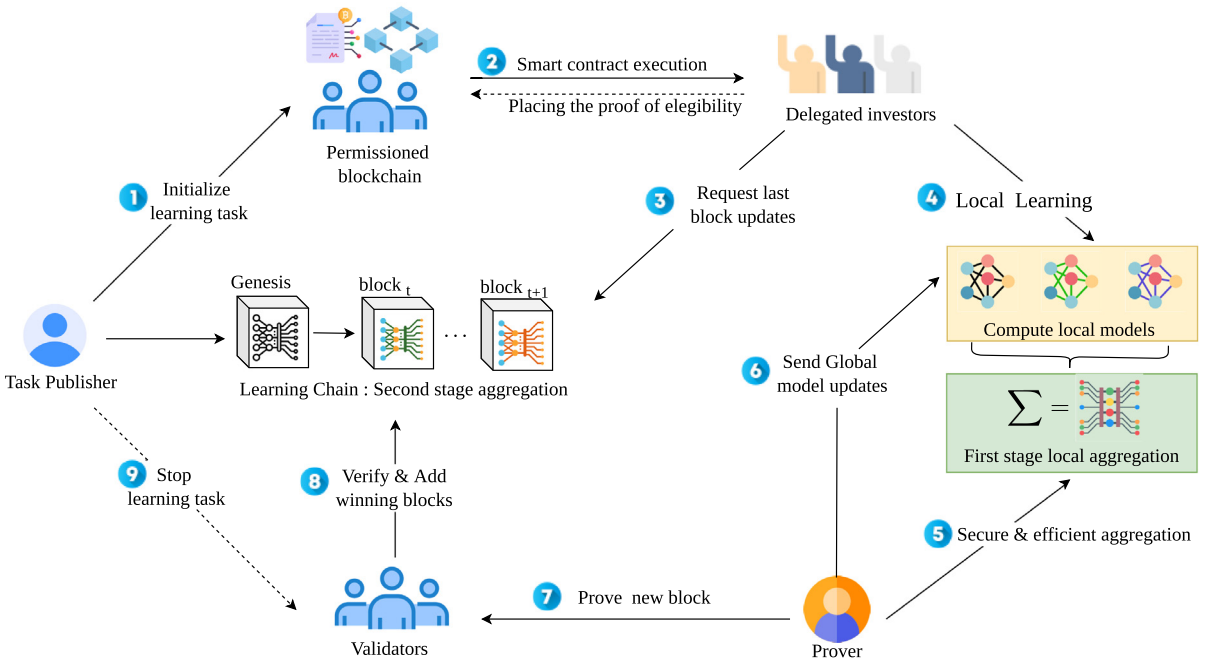


Fig. 1. An overview of our proposed blockchain and federated learning for Industrial IoTs.

across multiple industrial organizations as a result of strong data similarity, which is assumed to improve the overall model quality when decentralized resources are harnessed to improve model learning efficiency and quality. Firstly, to overcome FL security challenges, we adopt the permissioned blockchain to secure multi-party computation using authenticated and private peer-2-peer (P2P) channels between parties. We also design the blockchain to ensure model sharing between participants, which is used to incentivize the cross-silo (cross-organization) federation based on cryptocurrency as a reward mechanism. Secondly, we design two federated stages for model aggregation, where the first stage is a cross-device aggregation inside an organization, while the second stage is between participants using the blockchain as a model-containing blocks called *Learning-Chain*, thus leveraging the cross-silo federated transfer learning. In addition, we implement a validation process of local training results as a consensus mechanism for the blockchain named Proof-of-Federated deep learning *PoFDL*, which can provide improved privacy, reliability, and transparency. Fig. 1 presents an overview of our proposed blockchain and federated learning workflow between industrial organizations:

(1) The task publisher proposes the learning task (initial model, validation data, rewards, and conditions), using the Smart Contract (SC).
(2) Investors examine and apply (sign) for outstanding tasks after placing an initial proof of eligibility.
(3) The administrators delegate eligible investors with predefined terms and conditions, then granted them restricted access to the *Learning-Chain* according to their efficiency.
(4) Delegated investors start the FL task using their corresponding resources without data sharing.
(5) Each investor operate as *Prover* and moderates the FL task of corresponding devices by securing the aggregation, verifying transactions, and creating new blocks.
(6) A *Prover* sends global updates to corresponding devices for more federation rounds while not solving the PoFDL.
(7) If the PoFDL is solved, the *Prover* creates a new block and broadcasts it to the (*Validators*), who validate the block and reach consensus.
(8) If consensus is reached the block is added to the *Learning-Chain*, both the *Prover* and the *Validators* get corresponding rewards, and thus secure and ensure transfer learning between participants.

The task publishers and the provers could be internals of the blockchain system or externals. However, the validators are the trusted blockchain maintainers.

3.2. Preliminaries

3.2.1. Federated learning

Is one solution for distributed ML algorithms that allows training a global model using computational resources distributed across devices to provide data privacy. The main idea is to solve the conventional optimization problem of

ML iteratively in a distributed manner, such that each device tries to minimize its local cost function (also known as the loss function) through local training while seeking to optimize the global model parameters. Each round in FL consists of three main steps, namely: (1) device selection and sampling; (2) local computation; and (3) aggregation and consensus. FL optimization problem can be modeled as the following:

$$\min_{W \in \mathbb{R}^d} f(W) = \frac{1}{N} \sum_{i=1}^{N} \quad \mathfrak{I}(W, D_i) \tag{1}$$

Where $W \in \mathbb{R}^d$ denotes the global model parameters to be optimized, N, D_i denotes the participating devices and their corresponding training samples, and $\mathfrak{I}(W, D_i)$ denotes the cost function to be optimized and returns the locally computed updates. The consensus of participating devices is facilitated either with the help of a central entity called aggregation server, which describes the centralized FL, or by communicating with only neighboring devices which describes the decentralized FL [18].

3.2.2. Differential privacy (DP)

Is another theoretical model for privacy protection against anyone seeing the processing results of individual data [19]. It allows adding noise to sensitive data without significantly affecting the output of the analysis, which is quantified by the privacy parameters ϵ and δ. Formally, a randomized mechanism $M : \mathcal{D} : \rightarrow \mathbb{R}$ achieves (ϵ, σ)-DP if for any two adjacent datasets $d, d' \in \mathcal{D}$: where $d \ - \ d' \ \leqq 1$ (i.e. differ in at most at one sample) and for any output $\mathcal{O} \subset \mathbb{R}$, follows that :

$$\mathbb{P}[M(d) \in \mathbb{R}] \leqq \exp(\epsilon)\, \mathbb{P}[M(d') \in \mathbb{R}] + \delta \tag{2}$$

Where $\mathbb{P}[.]$ is the probability function, δ denotes the probability for fault tolerance, and ϵ denotes the privacy budget.

3.2.3. Permissioned-blockchain

A blockchain is a set of information-containing blocks that are connected linearly and secured with cryptography. Its power comes through decentralization, immutability, and shared permission to write to the system. Various algorithms have been used to verify and validate block data (i.e., transaction data), such as bilinear pairing, short signatures, hash functions, etc. Permissioned-Blockchains are one type of blockchain where all participants are known and reputable. To manage the integrity of transactions in permissioned-blockchains, we first need a standard approach to uniquely identify the participants in the network, we also need the authorization of transactions and verification that the content is not modified through digital signing. For that, a combination of hashing and public-key cryptography is required. Unlike public blockchains, permissioned blockchains have better scalability and flexibility, since they are fully controlled by a limited number of authorities and consensus can be easily reached. However, they can only partially ensure the nonrepudiation, transparency, and traceability of transactions [20].

3.2.4. Smart contracts (SC)

Are a fundamental technology of the blockchain that provides a powerful capability of code execution for embedding business logic (e.g. transaction fees, payment terms, enforcement, etc.) to enable agreements between distrusted parties [21]. However, they are exposed to confidentiality issues of blockchains. Recently a novel architectural scheme is proposed to enable confidentiality-preserving SC to safely compute sensitive data by separating consensus from execution by combining blockchains with Trusted Execution Environments (TEEs) [22].

3.2.5. Consensus mechanisms

In order for a participant (called a miner) to add a new block, he must provide proof of work (PoW) regarding his verified block. Then, a consensus mechanism will run between participants to validate block data and to ensure that the distributed ledger of the blockchain stays synchronized. Several consensuses have been proposed to replace hashing-based PoW with a low-overhead mathematical proof (i.e., a simpler protocol, easier to implement, check for correctness, and optimize for performance), such as Practical Byzantine fault tolerance (PBFT) [23], Proof of Authority (PoA) [24], etc. Recently, a novel proof of learning (PoL) mechanism was proposed in the context of DL to verify and validate the correct computation of model learning using either the training phase or the inference phase of the targeted model [25].

4. The proposed PPSS framework

This section presents the elements of the proposed PPSS framework, which includes the blockchain-enabled FL scheme and the privacy-preserving intrusion detection scheme. Table 1 presents the notation used in our PPSS framework.

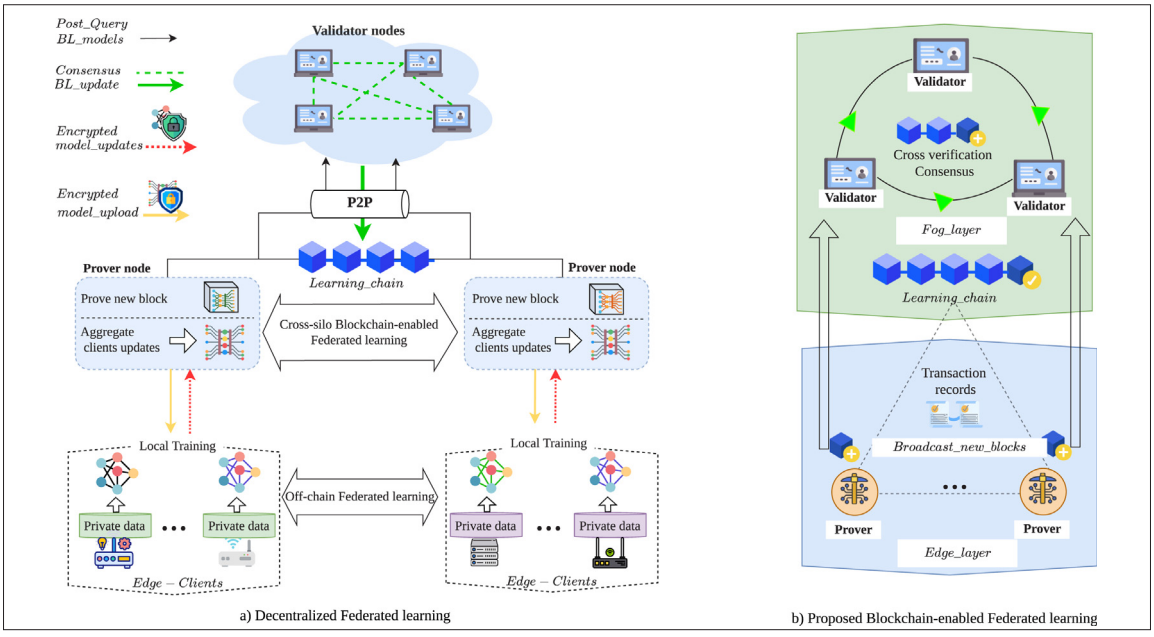


Fig. 2. The proposed security model for Industrial IoTs networks with an organizational chart of BL-enabling FL system.

4.1. Overview of Architectural Framework and system components

The proposed PPSS secure system is depicted in Fig. 2 and Algorithm 1, the main components are: *Trust authority*, *Validator* nodes $\mathcal{V}$, *Prover* nodes $\mathcal{P}$, and Edge-clients. We consider a *Trust authority* to achieve key management between blockchain participants, we consider N Edge-clients grouped according to property or location and connected to multiple $\mathcal{P}$ nodes. The goal of the system is to train a DL model with federated proceeding through a series of local and global aggregation using $\mathcal{P}$, and $\mathcal{V}$ nodes respectively. In the local aggregation, $\mathcal{P}$ uses confidentiality-preserving smart contracts [22] in order to aggregate corresponding client updates and prevent other nodes from exploiting their own updates. Then, the resulted aggregated model will be re-transmitted to clients for another communication round if this model is not qualified for the global aggregation. In the global aggregation maintained by $\mathcal{V}$, reported model-containing blocks from $\mathcal{P}$ will be added to the *Learning-Chain* using a consensus mechanism named Proof-of-Federated deep learning *PoFDL*. After that, each participant with the right to access the blockchain can request the last model stored in *Learning-Chain* and use it for deployment or for further improvement. In either case, it is required for those participants to stake a deposit of cryptocurrency on the blockchain, unless they submitted a new model, they get paid and have subsequent rewards. The interaction between PPSS components is summarized as follows:

- **Trust authority :** it introduces the single blockchain trusted authority in charge of launching the PPSS system by deploying smart contracts of submitted learning tasks and registering and generating cryptographic key pairs for all system nodes in order to establish secure communication channels.
- **Validator nodes :** present another trusted blockchain authorities and serves as validators when receiving new model-containing blocks to be added to the chain. They operate at the fog layer, the goal is to jointly maintain a blockchain and store new efficient models in the *Learning-Chain*. In order to reach a consensus for adding new models to the chain, these nodes use the algorithm of Proof of Authority (PoA). Once consensus is reached, each node will notify corresponding participants for transfer learning using the newly added models. We also consider that validator nodes can apply for outstanding tasks as provers using their own property of data and computing devices (Edge-clients).
- **Prover nodes :** these are server nodes that moderate the local FL between corresponding Edge-clients and ensure first-stage aggregation at the edge layer. They can both post new model-containing blocks and query information on the blockchain. Within an organization, a *Prover* $\mathcal{P}$ distributes an initial pre-trained model to corresponding selected devices and coordinates the FL process by aggregating and validating computed updates using a shared partition of the validation dataset. Secondly, at each federation round, if $\mathcal{P}$ receives model updates that meet a proof of efficiency named PoFDL from Edge-clients, it invokes the smart contract for global model aggregation by passing only authenticated updates. Then the smart contract returns the aggregated global model along with associated building transactions. After that, $\mathcal{P}$ will package all global model transactions onto a new block and send it to *Validator* nodes for verification and consensus. This would reduces network exchange and communication overhead.

- **Edge-clients :** Presents the authenticated FL devices where private data resides. When receiving a request for model updates from $\mathcal{P}$, these devices perform local and differentially private training before sending back computed updates to prevent privacy disclosure since one adversary might attempt to breach the privacy of training data by working only on the update. In order to enforce privacy-preserving, we adopt Abadi et al. [26] algorithmic technique for learning within a differentially private stochastic gradient descent algorithm (DP-SGD), where each edge-client adds locally a specific quantity of Gaussian noise to all gradients resulted from each training sample and after each local SGD steps. As described in Algorithm 1 (Edge-client 9–18), at each backward step of SGD, we compute an individual gradient $\mathbf{g}(x_i)$ for each data sample x_i. Second, since the gradient size has no prior bound, each gradient is clipped on $\breve{\mathbf{g}}(\mathbf{x_i})$ in the L_2 norm, as a result $\breve{\mathbf{g}} = \mathbf{g} \,/\, \max(1, \frac{\|\mathbf{g}\|_2}{C})$, where C denotes the gradient norm bound. After that, Gaussian noise is applied while all the gradients are merged into a single gradient before being used to update the parameter weights W.

Algorithm 1: PPSS secure aggregation

Validator nodes (*Learning_chain*):
  Validate submitted new model-containing blocks
  Consensus and adding model-containing blocks to the *Learning_chain*. Refer to algorithm 2
Prover nodes (*Learning_chain,* TestSet):
  Initialize model W from the *Learning_chain*
  for *each round* $t = 1, .., R$ **do**
    $S_t \leftarrow$ Subset a random set of clients k
    m $\leftarrow$ *Encrypt*(W, sh_k)
    Parallel.for $k \in S_t$ **do**
      $m_t, \sigma \leftarrow$ *Edge_client* (m, *Sign*$(m, \mathcal{P}sk)$)
      if *Verify* (m_t,σ) **then**
        $W^k_{t+1} \leftarrow$ *Decrypt*$(m_t.w_t, sh_k)$)
      end
    end
    $W_{t+1} \leftarrow$ ***aggregate***$(W^1_{t+1}, W^2_{t+1}, \ldots, W^k_{t+1})$
    Proof $\leftarrow$ predict(W_{t+1}, TestSet)
    if *Proof* $\geq$ *Learning_chain.Proof* **then**
      G_M, *Txs* $\leftarrow$ ***SC.aggregate***$(m^1, m^2, \ldots, m^k)$
      Submit new_Block(G_M, *Txs*)
    end
  end
Edge-clients (m, σ, *Params*):
  Params: α, E, ϕ, C
  Require: Local data : D; Associate $\mathcal{P}pk$
  Output: Privacy preserved, signed and encrypted model update
  /* For each client k do */
  if *Verify* (m,σ) **then**
    $W_k \leftarrow$ *Decrypt*$(m.w, sh_k)$)
  else
    Return
  end
  /* Differential private training */
  for *batch of samples* B_j *in [D]* **do**
    Compute gradient $\mathbf{g}$
    For each $i \in B_j$ compute $\mathbf{g}_j(x_i) \leftarrow \partial_{w_j} \mathbb{J}(w_j, x_i)$
    Clip gradient $\breve{\mathbf{g}} \leftarrow \mathbf{g}$
    $\breve{\mathbf{g}}_j(x_i) \leftarrow \mathbf{g}_j(x_i) \,/\, \max\,(1, \frac{\|\mathbf{g}_j(x_i)\|_2}{C})$
    Add noise
    $\tilde{\mathbf{g}}_j \leftarrow \frac{1}{|B|}\,(\sum_i\,(\breve{\mathbf{g}}_j(x_i) + \mathcal{N}(0, \phi^2, C^2\mathbf{I})$
    Decent
    $w_{i+1} \leftarrow w_i - \alpha \,.\, \tilde{\mathbf{g}}_j$
  end
  m $\leftarrow$ *Encrypt* (W_E, sh_k)
  Send (m, *Sign*$(m, \mathcal{C}sk)$) to correspond Prover node

Table 1
Notation for the discussion of algorithms.

Notation	Description
C	Gradient norm bound
I	The identity matrix
D	Local data
E	Local epochs
k	Global security parameter
m	Message
W	Model weight
σ	Signature
SC	Smart Contract
sh_k	Ephemeral symmetric key
$\mathcal{P}sk$	Prover secret key
$\mathcal{P}pk$	Prover public key
$\mathcal{C}sk$	Client secret key
$\mathcal{C}pk$	Client public key
α	learning_rate
ϕ	Noise scale
(ϵ, δ)	Privacy cost
$g(x_i)$	Gradient computed on x_i
G_M	Global model
Txs	Building transactions of global model
$aggregate(.)$	Aggregate models by averaging them (FedAvg)
$SC.aggregate(.)$	Aggregate models using smart contract
$Sign(.)$	Digital signature function
$Verify(.)$	Verify digital signature function
$Encrypt(.)$	Symmetric encryption function
$Decrypt(.)$	Symmetric decryption function
T_x	Transaction
Proof	Performance metric such as Accuracy
TestSet	Cross-validation dataset

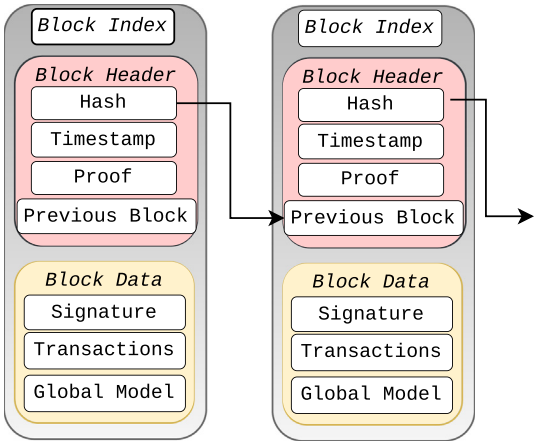


Fig. 3. Structure of proposed blockchain used by PPSS framework.

4.2. Blockchain based scheme

We adopt the permissioned blockchain business model, where only pre-selected nodes called *Validator* nodes can participate in consensus and generate blocks. We design the BL scheme as a *Learning-Chain* for updating and sharing models using secure P2P communication channels and an energy-efficient consensus mechanism named Proof-of-Federated-Deep-Learning (PoFDL). All transactions with the *Learning-Chain* such as posting and querying a model are recorded in a distributed ledger and only allowed to relevant participants (validator and prover nodes) who can operate on the blockchain system. The PoFDL is a tuple of algorithms : $(PSetup\left(1^k\right), KeyGen\left(\mathcal{E}\right), KeyAgGen\left(\mathcal{E}, \mathcal{A}\right), Sign\left(sk_i, m\right), Verif\left(m, pub_i, \sigma\right))$ where:

- $PSetup(1^k)$: is the algorithm that takes 1^k where k is the security parameter of the system and returns the description of bilinear groups $\mathcal{E} = (p, G_1, G_2, G_T, e)$.

- *KeyGen*($\mathcal{E}$): is the algorithm that selects two generators $g_1 \in G_1$ and $g_2 \in G_2$ with a random scalar $x \leftarrow Z_p$. Then, it produces a public/private key pair (pk_i, sk_i) for the party invoking it, where $pk_i = (g_1, g_2, \tilde{X}, z)$, $\tilde{X} \leftarrow {g_1}^x$, $z \leftarrow e(g_1, g_2)$, and $sk_i = x$.
- *KeyAgGen*($\mathcal{E}, \mathcal{A}$): is the algorithm that selects a sequence of public keys $\mathcal{A}$ and then produces an aggregate public/private key pair ($\mathcal{A}pk_i, \mathcal{A}\ sk_i$).
- *Sign*(sk_i, m): is the signature algorithm, that takes a message $m \leftarrow [WR, TS, AUX\ ]$, a private key sk_i and produces a signature $\sigma \leftarrow g_1^{\frac{1}{x+m}}$, *AUX* includes the training information (e.g., hyper-parameters), *TS* is an optional collection of test data samples to evaluate model updates, and *WR* is a collection of intermediate model weights registered during the training process.
- *Verif* (m, pub_i, σ): is the verification algorithm, where σ is a signature, $m \leftarrow [WR, TS, AUX\ ]$ is a message, pub_i is a public key. This algorithm tests $e\left(\sigma, \tilde{X} \cdot {g_2}^m\right) = z$ and returns true or false.

Definition 1 (*PoFDL Correctness*)**.** Let O be a t-PoFDL scheme initialized with $\mathcal{E} \leftarrow PSetup(1^k)$, *KeyGen*($\mathcal{E}$), and *KeyAgGen*($\mathcal{E}, \mathcal{A}$), where $\mathcal{E} = (p, G_1, G_2, G_T, e)$. Let ($pk_1, sk_1$), $\cdots$, (pk_i, sk_i) be a sequence of keys generated via *KeyGen*($\mathcal{E}$). Let ($\mathcal{A}pk_i, \mathcal{A}\ sk_i$) be an aggregate public/private key pair generated via *KeyAgGen*($\mathcal{E}, \mathcal{A}$). Let $m \leftarrow [WR, TS, AUX\ ]$ be a message and let (pk_1, σ_1), $\cdots$, (pk_i, σ_i) be any sequence of key/signature pairs, where $\sigma \leftarrow g_1^{\frac{1}{x+m}}$. The O scheme is valid if, for every message and sequence, the following criteria are satisfied:

- When the *Verif* (m, pub_i, σ_i) algorithm tests $e\left(\sigma_i, \tilde{X} \cdot {g_2}^m\right) = z$, the result is true for all i;

The proposed BL-enabled scheme contains four different phases: (1) Initialization phase, (2) agreement phase, (3) Generating a model-containing block phase, and (4) Consensus-making phase.

- Initialization phase: This includes the registration of all PPSS nodes by the *Trust authority* ($\mathbb{A}$) : the *Validator* nodes ($\mathcal{V}$) as trusted parties who can validate and add new blocks, the *Prover* nodes ($\mathcal{P}$) who can only query the BL and generate new blocks, and *Edge-clients* owned and controlled by *Prover* nodes as model trainers without the ability to query the *Learning_Chain*. Each entity is given the following information: a blockchain-registered unique $\mathbb{ID}$ for authentication and a security parameter k for key generation. Each $\mathcal{P}$ node obtains the initial model state W_0 to be improved, learning_parameters, a partition of Validation_data for building the PoFDL from the *Learning_Chain*. *KeyGen*($\mathcal{E}$) $\rightarrow$ (pk_i, sk_i) allows $\mathcal{V}$ nodes to generate a private–public key pair. It also allows *Edge-clients* as model trainers to generate a private–public key pair. *KeyAgGen*($\mathcal{E}, \mathcal{A}$) $\rightarrow$ ($\mathcal{A}pk_i, \mathcal{A}sk_i$) allows *Provers* to generate a private–public key pair from corresponding Edge-clients public keys $\mathcal{A}$. Each $\mathcal{P}$ node loses its credential and permissions if the qualified models are not submitted according to smart contract conditions.
- Agreement phase: When a participant publishes a learning task by providing the initial information and conditions (i.e. initial model state W_0, labeled test_dataset TestSet, parameters) for provers who want to join the federated learning task. A Prover $\mathcal{P}$ must provide proof of eligibility such as training performance using its own resources to the smart contract terms in order to contribute in *Learning-Chain*. The selected eligible provers must register all corresponding Edge-clients to ensure authentication.
- Generating a model-containing block : a Prover $\mathcal{P}$ generates a block with corresponding Edge-clients updates. The latter are verified by using the tamper-proof ledger of the *Learning_chain* as a reference to identify malicious clients. Then only valid updates will be encapsulated as transactions (T_x). Given two different hash functions : $H_1 : \Theta \times \{0, 1\} \rightarrow \{0, 1\}$ and $H_2\{0, 1\} \times \{0, 1\}^* \rightarrow \Omega$. Given a secret key $x_i y_i \in \mathcal{Z}_p^*$ and a block $Bloc_i \in \{0, 1\}^*$, $\mathcal{P}$ picks : $g_1 \in G_1$, a random number $r_i \in \mathcal{Z}_p^*$ and computes $\sigma_i = H_1(g_1^{\frac{1}{x_i + Bloc_i + y_i + r_i}}) \in \{0, 1\}$. Then $\mathcal{P}$ computes $b_i = H_2(\sigma_i, Bloc_i, r_i) \in \Omega$ and sets the time intervals of a block generation as T, The signature of a block $Bloc_i$ is (σ_i, b_i, r_i). Finally $\mathcal{P}$ broadcasts transaction data combined with the signature to the blockchain *Validator* nodes.
- Consensus-making phase: We adopt the Proof-of-Federated Deep-Learning (PoFDL) that is inspired by Proof-of-Authority (PoA) consensus mechanism to complete verification and add new blocks to the *Learning-Chain*. However, instead of just relying on pre-selected reputable nodes, we consider that each *Prover* has the right to become a *Validator* node in the PoFDL by staking a deposit of cryptocurrency or staking his reputation. Our idea will increase the trust level across participant and enhances blockchain immutability. The PoFDL validator nodes are the trusted parties that operate as mining authorities for maintaining the blockchain by adding new blocks. Once a *Prover* generates a new block, it sends it to the corresponding mining authority for verification. This authority operates as "**Leader**" for the next block in the *Learning-Chain*. To fairly distribute the responsibility of block creation amongst validators, PoFDL adopts a time-based mining rotation scheme and ensures one elected **Leader** (validator node) at each time-step through the smart contract [27]. If the current leader does not send a block during the elapsed time, an empty block that would undermine his reputation has to be sent. Fig. 4 depicts the consensus process and message exchanges of block proposals based on PoFDL. The *Leader* broadcasts received block to the other validators for block acceptance. Each *Validator* ($\mathcal{V}$) evaluates the received model-containing block, broadcast results, and compares them to those of other validators to determine block acceptance. The process of block validation and the consensus are depicted in Algorithm 2. The block is added to the chain if: (1) the *Leader* is the one expected to be the current leader. (2) At least $\frac{N}{2} + 1$ *Validators* received the same block and confirm with acceptance.

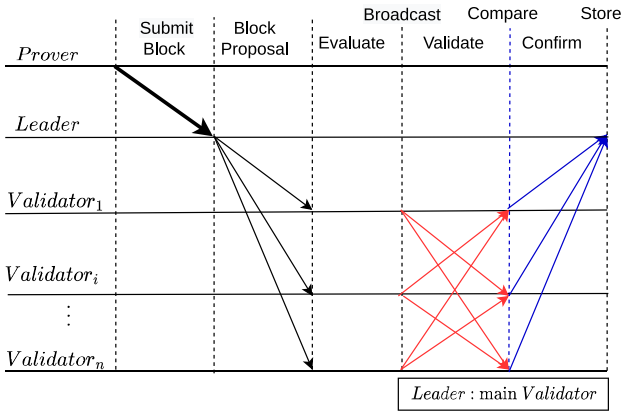


Fig. 4. Consensus and message exchange of proposed PoFDL protocol.

Algorithm 2: Validators : Validate and add new block

Input: Block, previous_hash, Dependent Validation data : $\mathbb{D}^*$
Output: Add valid block : Success or Fail

For Validator $\mathcal{V} \in N$ **do** ;
if *Sender is* ***current*** *Leader* **then**
  if *Authenticated and Valid (Block.signatures)* **then**
    Valid_proof $\leftarrow$ predict(Block.model, $\mathbb{D}^*$);
    if *Average(Valid_proof , Block.Proof)* $\geq$ *Block[previous_hash].Proof* **then**
      $\mathcal{V}$ Broadcast m(Block.hash, **Success**) to Validators
    end
  else
    Broadcast to all (**Block.hash, Fail**)
  end
else
  Broadcast to all (**Block.hash, Fail**)
end
if *Each* $\mathcal{V} \in N$ *upon receiving at least* $N/2 + 1$ *m(Block.hash,* ***Success****)* **then**
  Reaches consensus and Confirm adding full Block data to Leader
else
  Sends a penalty to Leader
end
If Leader upon receiving at least $N/2 + 1$ Confirm adding (Block.hash), it stores current block.

Unlike the prior concept of proof of learning [25], our PoFDL uses the inference phase to validate the correctness of learned models, where the cost of inference is much lower, and it does not leak any sensitive data in order to check or validate the proof. Specifically, we consider that : (i) each *Prover* $\mathcal{P}$ is given the same TestSet of labeled data to prove new models, (ii) each *Validator* $\mathcal{V}_i \in N$ is given a dependent and identically distributed partitions of labeled data $D_i \in$ ValidationSet at the initialization phase by the task publisher, where:

$$\text{TestSet} \bigcap \text{ValidationSet} = \phi \tag{3}$$

$$\text{ValidationSet} = (D_1, D_2, \ldots, D_N) \tag{4}$$

$$\forall (D_i)_{i \in I} \subset \text{ValidationSet};\ \text{with } Card\ I = \frac{N}{2};\ \bigcap_{i \in I} D_i = \phi \tag{5}$$

We require that at least $N/2$ validation data partitions are disjoint sets to prevent *Provers* and Byzantine nodes from cheating by training on this data and submitting invalid models. As illustrated in Fig. 4, $\mathcal{P}$ is assigned to the current *Leader* node of the *Learning_chain*. If $\mathcal{P}$ has the privilege and operates as $\mathcal{V}$, then it will await his turn to operate as a *Leader*. A *Leader* proposes verified model-containing blocks to other *Validator* nodes. After the latter evaluates the received block data and broadcasts their results (Success or Fail) with their signatures to each other, each node compares its result with others for consensus. If at least $\frac{N}{2}+1$ *Validator* nodes agree on the block, then the consensus is reached and a confirmation

message is sent as a replay back to the *Leader* node. After that, this block is stored in the *Learning_chain*, and both the *Leader* and the *Prover* are awarded by cryptocurrency.

In terms of effectiveness, the PPSS framework greatly reduces computational complexity, energy consumption, communication overhead, and network load. In addition, the PPSS framework leverages both cross-silo and cross-device FL to optimize transfer learning and jointly maintain high-quality models. Hence, the system is robust, secure, and tolerant to the $N/2$ maximum number of byzantine nodes.

4.2.1. Secure communication and key management

To enable secure end-to-end communication between proposed PPSS system nodes, we propose a combination of asymmetric and symmetric encryption algorithms, employing RSA with the advanced encryption standard (AES) cryptosystem. The AES algorithm is used to effectively encrypt data using a shared ephemeral key, while the RSA algorithm is used to secure authentication using a private–public key pair through digital signatures. To achieve secure key management, we propose a central "*Trust authority*" ($\mathbb{A}$) in charge of generating a private–public key pair for each system node. The public keys are stored as identities ($\mathbb{ID}$s) and broadcast to all *Validator* ($\mathcal{V}$) and *Prover* ($\mathcal{P}$) nodes, and the private keys are given to the corresponding nodes. In the second-stage aggregation, $\mathbb{A}$ is also in charge of establishing a shared AES ephemeral key between $\mathcal{P}$ and $\mathcal{V}$ nodes to ensure secure communication. However, in the first-stage aggregation, $\mathcal{P}$ has the role of securing the communication between correspondent *Edge-clients*. Where each $\mathcal{P}$ is responsible for generating and safely exchanging an ephemeral key for AES-based data encryption. This would limit the attack surface against the maintained *Learning-Chain* as well as secure the model transmitting from exposure to eavesdroppers and spyware.

4.2.2. Security analysis

- Byzantine and Sybil attacks: Adversarial participants may negatively affect the performance of the global model as well as the *Learning_chain* consistency, by increasing their influence in the systems. To prevent this influence, the blockchain authorities would only admit *Provers* with reputations earned by positively contributing to the learning environment being both *Leaders* and *Validators*. In addition, our proposed consensus protocol allows trusted *Validators* to collaboratively detect malicious nodes during their leadership using a voting mechanism. A leader node can be voted out as malicious according to the following situations: (1) if it has not proposed any block; (2) if it has proposed more blocks than expected (Denial of Service attacks), or (3) if it has proposed different blocks to different authorities [27].
- Model inversion and membership inference attacks [28,29]: Besides the encryption and authentication mechanisms that protect model sharing from the public, privacy breaches may occur in honest-but-curious scenarios, when an adversary tries to determine whether a given data record was part of the model's training dataset or not, since all the *Learning_chain* participants have full access to model parameters. In our PPSS framework, we consider adding a differential privacy module as a preventive measure against these types of attacks. Specifically, we differentiate parameter gradients of client models during training in order to preserve the privacy of training data.
- Model theft attacks: It refers to the possibility of a consensus node stealing a trained model when it is received for validation from other Provers, then claiming ownership by re-broadcasting it to other consensus nodes, as in the replay attack [30]. To prevent these attacks, firstly, we require that a *Prover* node must include intermediate clients' updates as building transactions of the global model in the block data (Fig. 3). These transactions are timestamped and hashed using corresponding and registered client IDs in the blockchain. Secondly, we require that a *Prover* also include a signature in the block data that contains his ID and block data. This made it difficult for an adversary to spoof block data and re-broadcasting it. In addition, our proposed idea would also reduce the communication overload of message exchanges between *Prover* nodes and the *Validator* nodes for investigation.

4.3. The intrusion detection scheme

To protect industrial networks from large-scale and emergent cyber attacks such as DDoS attacks against the availability of resources, backdoor attacks against privacy breaches, or even a legitimate exploit but for external benefit, we further propose a PPSS-enabled and anomaly-based IDS. Besides privacy and security, the PPSS framework also enabled a decentralized deployment of the IDS system, where detection nodes are frequently updated with efficient and reliable detection models, trained over large networks with the minimum cost. In addition, anomaly-based detection is proven to be a very efficient and self-adaptive method to detect unknown attacks (i.e. Zero-day attacks), which enables the IDS to trace and analyze different behaviors (e.g., network traffic, used operating systems or devices) in order to detect any potential and malicious activity on time. For data source, we use our recently proposed data-set named Edge-IIoTset [31]. The data-set was collected from realistic industrial traffic data, which was generated from different IoT devices and overall a network of seven interconnected layers with related protocols, including, cloud computing layer, NFV layer, Blockchain layer, fog layer, SDN layer, edge layer, and IoT/IIoT perception layer. Furthermore, it contains fourteen labeled network attacks besides legitimate traffic data.

Unlike previous work in which we used a deep neural network (DNN) for intrusion detection and demonstrated its superiority against ML-based approaches [31], this study uses the convolutional neural networks (CNN) as a detection module. The CNN is another popular DL approach and a dedicated class of neural networks for data processing with a

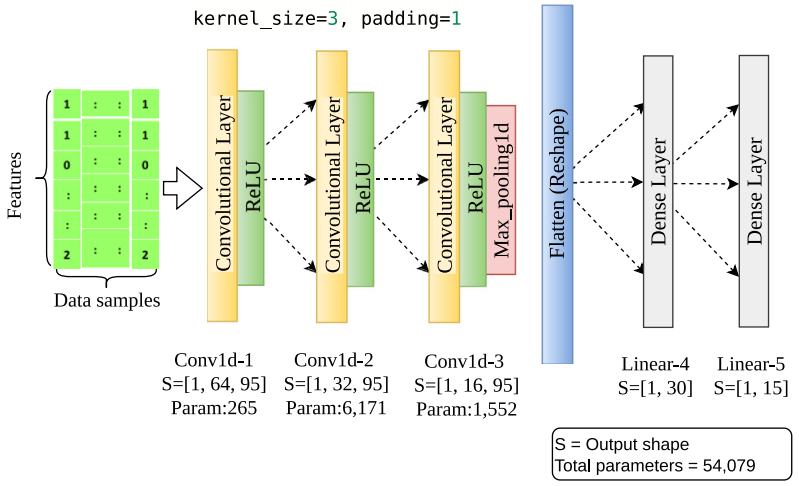


Fig. 5. Structure of CNN model adopted by PPSS framework.

Table 2
Experimental settings for federated learning.

	Parameter	Values
Fed_Learning	Client	[20,40,80]
	Global rounds	15
Differential privacy	Epsilon (ϵ)	[0.1, 1, 2]
	Delta (δ)	1.5e−5
	Gradient norm bound (C)	1.2
*	Model	CNN() Refer to 5
	Learning rate	0.01-0.001
	Optimizer	Adam
	Loss_function	CrossEntropyLoss()
	Local epochs	1
	Local Batch_size	100
	Learning rate	0.01

familiar network structure, designed mainly to extract discriminatory spatial features for model decisions. It is composed of a set of convolutional layers that use a mathematical operation called convolution and processing perceptron layers, which are well equipped to treat and handle large-scale cyber attacks. [32]

Fig. 5 illustrates the structure of the proposed CNN model. Several experiments were carried out to obtain an accurate and generalized detection model as well as to validate the efficiency of the PPSS framework.

5. Performance evaluation and analysis

5.1. Experimental design

The experiment results of the proposed PPSS framework were performed on Google Colaboratory free environment, using PyTorch library and Tesla-T4 GPU hardware accelerators. We have implemented the PPSS-enabled decentralized FL (DFL) using two stages of aggregation. Firstly, we have implemented the localized FL using multiple edge Provers (*i.e. FL servers*), each with corresponding resources of data and clients. Then we experimented with different scenarios for DFL such as the number of participating clients for each prover and differential privacy training within DP-SGD, in order to evaluate the first aggregation stage when using limited data as well as when considering privacy-preservation. Secondly, we have implemented the DFL that uses the proposed PoFDL in order to ensure transfer learning between all participating provers (i.e. second-stage aggregation). At this stage, the DFL performs a restricted federated aggregation using only validated models stored in the *Learning_chain*. Consequently, we have evaluated the global model performance considering both types of data distribution (IID/Non-IID, ref. 5.2) and data augmentation regarding the number of participating *Provers*. In the differential privacy settings, we used Opacus library [33] for implementation with a noise multiplier (ϵ, δ) and a maximum gradient norm value of $C = 1.2$. Table 2 describes the experimental settings and learning parameters used for this study. Tables 5 and 6 summarize the evaluation results of both centralized FL (CFL) and DFL in different settings. In order to measure how may security constraints affect the learning process, we have used different metrics to evaluate

both CFL and DFL-based IDS detection efficiency and effectiveness in terms of Accuracy, Precision, Detection Rate, Time complexity, and Energy cost.

- *Accuracy (Acc)*: given by:

$$Acc = \frac{TP + TN}{TP + FP + TN + FN} \quad \text{where :} \tag{6}$$

TP: denotes the number of correctly categorized positive samples.
TN: denotes the number of correctly categorized negative samples.
FP: denotes the number of wrongly categorized positive samples
FN: denotes the number of wrongly categorized negative samples.

- *Precision (Pr)*: denotes the proportion of proper attack classifications (TP) attack predictions to the total amount of predicted attack results and given by :

$$Pr = \frac{TP}{TP + FP} \tag{7}$$

- *Detection rate (Dr)*: denotes the proportion of proper attack classifications (TP) relative to the overall count of all samples that ought to have been identified as attacks and given by :

$$Dr = \frac{TP}{TP + FN} \tag{8}$$

- *Time complexity*: denotes the time complexity of the global model convergence and depends on the client's training time, model aggregation, and PoFDL consensus inference time. The time complexity of secure communication and transmission was not considered.
- *Energy cost*: denotes the energy consumption of the total training of global model, and given by [34] :

$$\Theta(e, \mathcal{N}, \mathcal{R}) = \sum_{r=1}^{\mathcal{R}} \sum_{i=1}^{\mathcal{N}} \mathbf{1}_{\{n_{i,r}\}}(t_{n_i}, e_{n_i}) \quad Kwh \tag{9}$$

Where, e denotes the energy power of $\mathcal{N}$ devices, $\mathcal{R}$ denotes the total federated rounds, t_{n_i} denotes the wall clock time of a device n_i at round r, e_{n_i} the energy power of n_i at round r, and $\mathbf{1}_{\{n_{i,j}\}}$ is the indicator function testing if a device n is selected for FL training at round r (n_i could be a client or a server). The energy cost is expressed by Kilowatt per hour (*Kwh*) and estimated using *Carbontracker* library [35].

5.2. Heterogeneity in data distribution (IID and Non-IID)

Data distribution is a very important feature of FL. In order to validate training performance using heterogeneous data distribution amongst participants, we divided the training set into independent and identically distributed (IID) partitions and allocated them to groups of clients, each group is managed by a Prover node (i.e, server). However, in the Non-IID scenario, we implemented a label partition method to have each client group have a random subset of labels with the same feature vector of training data, assuming that each Prover node knows partially the total classes of the problem.

5.3. Numerical results

Several experiments have been conducted to evaluate the efficiency of the proposed PPSS-enabled Decentralized FL (DFL) in order to measure how may security constraints affect the learning process.

5.3.1. Per-class performance using different models

Table 3 presents per-class performance results using the obtained global models from the different design schemes of FL (CFL and DFL) as well as the performance compared with data distribution and DP-training settings. We can see that both CFL and DFL have achieved 100% of precision and detection rate in identifying the normal traffic in all experiments. This goes back to the availability of benign traffic for all participants. For attack identification, we notice that training over Non-IID data and IID mostly gives the same results in precision and detection rate for both CFL and DFL. We can determine that federated aggregation can efficiently ensure transfer learning between participants with Non-IID data.

When introducing the differential privacy training using DP-SGD, we can see that the latter had negatively affected the overall attack detection performance for both CFL and DFL, especially for weakly represented classes that are too sensitive to small model weight changes. We can determine that introducing noisy gradients in the training phase to preserve privacy would disturb the final model decisions if we do not consider model sensitivity to small weight changes. In general, the DFL outperforms the CFL in identifying attacks due to that DFL performs one additional aggregation phase using only the qualified model stored in the *Learing_chain*. We can see that DDoS_ICMP and DDoS_UDP attack classes are well identified with a detection rate between 90%–100% in all settings. However, for Fingerprinting and Ransomware attacks, they were largely misclassified. We can identify the limitation of transfer learning when data is insufficient, which would result in significantly poor classification for minority classes, particularly in FL settings.

Table 3
Per-class performance using different models.

Classes	Metrics	IID data distribution				Non-IID data distribution				Support
		Precision %		Detection rate%		Precision%		Detection rate%		
	Settings	CFL	DFL	CFL	DFL	CFL	DFL	CFL	DFL	
Normal	No-DP	100	100	100	100	100	100	100	100	323129
	DP	100	100	100	100	100	100	100	100	
Backdoor	No-DP	72	74	90	89	68	76	95	88	4972
	DP	72	72	89	89	40	00	92	00	
Vulnerability_scan	No-DP	93	94	85	85	94	94	85	85	10022
	DP	93	93	85	77	93	93	84	84	
DDoS_ICMP	No-DP	100	100	100	100	100	100	97	100	23287
	DP	100	94	97	100	100	90	69	100	
Password	No-DP	43	100	83	07	100	10	07	07	10031
	DP	14	36	01	100	00	00	00	00	
Port_Scanning	No-DP	65	65	09	09	29	65	92	09	4513
	DP	00	00	00	00	32	00	10	00	
DDoS_UDP	No-DP	98	98	100	100	98	98	100	100	22007
	DP	93	100	100	99	58	98	100	100	
Uploading	No-DP	59	57	39	37	31	100	83	15	7527
	DP	00	100	00	00	00	00	00	00	
DDoS_HTTP	No-DP	71	70	99	99	71	75	97	94	9982
	DP	70	67	99	99	70	70	98	99	
SQL_injection	No-DP	54	41	17	90	42	39	28	100	10241
	DP	37	00	98	00	37	37	100	100	
Ransomware	No-DP	00	00	00	00	00	77	00	14	2185
	DP	00	00	00	00	00	00	00	00	
DDoS_TCP	No-DP	69	68	100	100	00	69	00	100	10012
	DP	37	68	100	100	00	53	00	100	
XSS	No-DP	92	100	05	02	65	52	10	28	3183
	DP	00	100	00	00	33	00	00	00	
MITM	No-DP	100	100	100	93	100	100	100	93	80
	DP	00	00	00	00	00	00	00	00	
Fingerprinting	No-DP	00	00	00	00	13	00	57	00	200
	DP	00	00	00	00	00	00	00	00	

CFL : Centralized federated Learning; **DFL** : Decentralized federated Learning;
No-DP : No differentially private training; **DP** : with differentially private training; **Support** : number of test samples.

5.3.2. The evaluation results DFL federated learning in IID

Table 5 presents a comparison between global model accuracies of proposed DFL under different settings (i.e., number of clients distributed over multiple provers, DP settings, and data distribution). We selected the test accuracy to determine the best, the worst prover accuracy achieves for all client distribution sets and at the first and the 15th round of DFL. In the IID mode, when 3 provers are contributed to updating the global model, the best prover accuracy achieves 93.71% with $K = 40$, and the worst prover accuracy achieves 92.80% with $K = 20$, while the accuracy of the global model reached 93.99% with $K = 20$. However, in the differential privacy settings, the best prover accuracy achieves 92.65%, the worst prover accuracy achieves 90.31% with $K = 80$, while the accuracy of the global model reached 93.40% with $K = 20$. When 6 provers are contributed in updating the global model, the best prover accuracy achieves 93.83%, while the accuracy of the global model reached 93.98% with $K = 20$. In the differential privacy settings, the best prover accuracy achieves 93.46% with $K = 40$ while the accuracy of the global model reached 93.72% with $K = 80$. When 8 provers are contributed to updating the global model, the best prover accuracy achieves 93.86% with $K = 20$, while the accuracy of the global model reached 94.01% with $K = 40$. In the differential privacy settings, the best prover accuracy achieves 93%, the worst prover accuracy achieves 90.92% with $K = 20$, while the accuracy of the global model reached 93.36% with $K = 40$.

5.3.3. The evaluation results DFL federated learning in Non-IID

Table 6 presents a comparison between global model accuracies of proposed DFL under different settings (i.e., number of clients distributed over multiple provers, DP settings, and data distribution). In the Non-IID mode, when 3 provers are contributed to updating the global model, the best prover accuracy achieves 92.45% with $K = 80$, the worst prover accuracy achieves 81.36% with $K = 40$, while the accuracy of the global model reached 92.52% with $K = 80$. However, in the differential privacy settings, the best prover accuracy achieves 89.55%, the worst prover accuracy achieves 80% with $K = 20$, while the accuracy of the global model reached 91.74% with $K = 20$. When 6 provers are contributed to updating the global model, the best prover accuracy achieves 93.27%, the worst prover accuracy achieves 83.22% with $K = 20$, while the accuracy of the global model reached 93.75% with $K = 40$. In the differential privacy settings, the best prover accuracy achieves 92.11%, the worst prover accuracy achieves 80.29% with $K = 80$, while the accuracy of the global model reached 93.18% with $K = 40$. When 8 provers are contributed to updating the global model, the best prover accuracy achieves 93.40% with $K = 80$, while the accuracy of the global model reached 93.74% with $K = 20$. In the

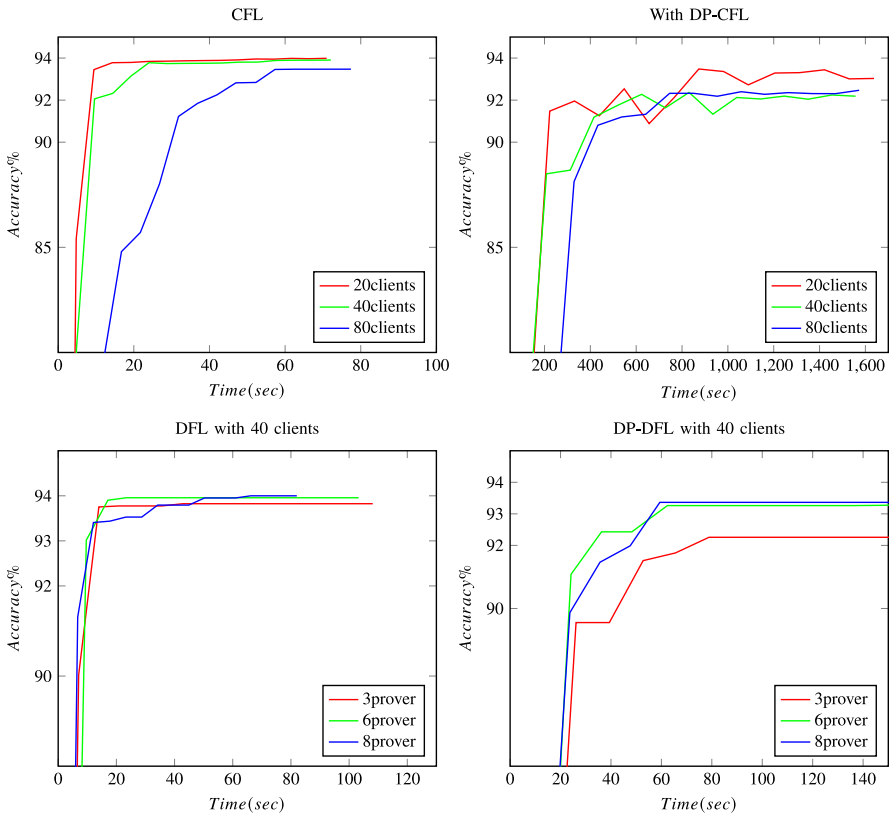


Fig. 6. Comparison of global models accuracy over time using number of provers and clients. DP-CFL: Centralized-FL using DP-SGD for training; DP-DFL: Decentralized-FL using DP-SGD for training.

differential privacy settings, the best prover accuracy achieves 91.71%, the worst prover accuracy achieves 80.84%, while the accuracy of the global model reached 92.63% with $K = 80$.

We can demonstrate that increasing the number of participating provers in Non-IID mode would enhance the overall model accuracy, and efficiently ensures knowledge exchange of various attack classes' representation. Consequently, the global model would generalize and perform well in detecting large-scale cyber attacks. Furthermore, we can demonstrate that by including a number of provers, the negative impact of differential privacy on model accuracy can be mitigated.

5.3.4. The performance of the global model accuracy according to the number of provers and clients

Fig. 6 illustrates a comparison of the global model's accuracy over time using the number of provers and clients. We can see how the DFL outperforms the CFL in terms of global model convergence time especially when training with DP-SGD. This is due to the fact that the DFL employs fewer training iterations but higher model aggregation operations than the CFL. As an example when deploying N clients in the CFL, the DFL deploys $\frac{N}{P}$ (where P is the number of provers) clients for each prover for training, while ensuring transfer learning for all N clients through aggregation. We can demonstrate that the cost of model aggregation is much lower than the cost of training. Furthermore, we can determine the efficiency of the DP-training in DFL by incorporating more provers (i.e., adding more training data), which would alleviate the DP effects.

5.3.5. The effect of differential privacy training using DP-SGD on global model accuracy

Training with DP-SGD for privacy preservation is computed using the strong composition theorem which states that the privacy leakage modeled through standard (ϵ, δ)-differential privacy grows approximately in $\sqrt{K}$ for high privacy regimes, where K is the number of training iterations as depicted in Figs. 7 and 8. Fig. 7 illustrates the effect of differential privacy cost (ϵ, δ) on the global model performance in both data distribution modes (IID, Non-IID). We have used different values of epsilon ($\epsilon = 0.1, \epsilon = 1, \epsilon = 2$) (i.e., the level of the introduced noise) with fixed $\delta = 1.5e - 5$ for both CFL and DFL training approaches, in order to demonstrate the trade-off between data privacy and model performance, and also to demonstrate the practicality of DP in Non-IID settings. The reason we fixed δ is that: according to [26], both ϵ and δ have similar effect on the introduced noise. However, ϵ parameter has the largest effect on the training performance. We can see that the CFL had an important decrease in performance. For example, for $\epsilon = 1$ the accuracy of CFL dropped from

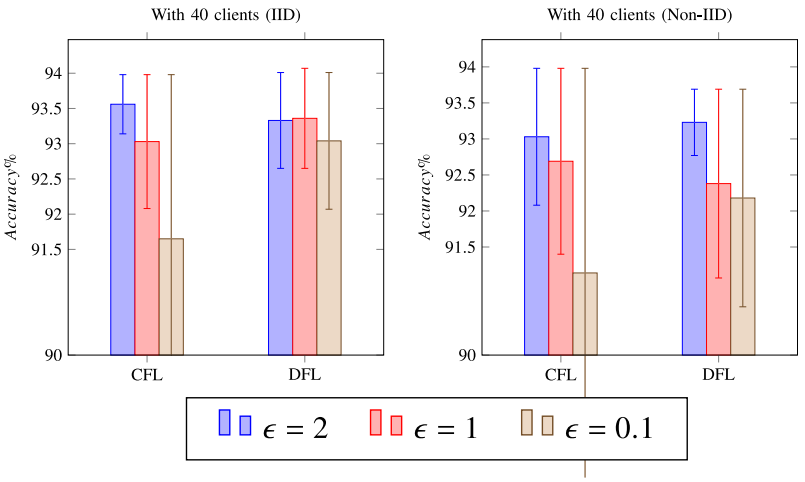


Fig. 7. Comparison of global model performance using DP-SGD with high privacy regimes.

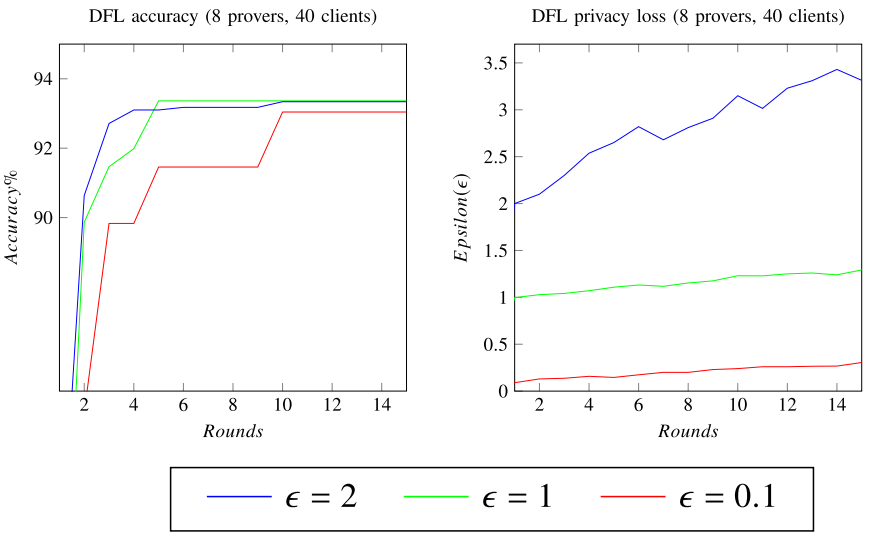


Fig. 8. Comparison of global models training and privacy loss for different noise levels.

Table 4
The average rate of data generation and the storage overhead of the *Learning-Chain*.

Metric	3 Provers	6 Provers	8 Provers
Nb_transactions	195	226	159
Nb_Blocks	6	9	5
Storage (MB)	43.48	50.84	35.48

93.98% to 92.69% for $\epsilon = 1$ and to 91.14% for $\epsilon = 0.1$. However, the accuracy of DFL dropped from 94.01% to 92.38% for $\epsilon = 1$ and to 92.18% for $\epsilon = 0.1$. This is a well-known consequence of the introduced noise and eventually results in a trade-off between model performance and data privacy protection. However, we can demonstrate that our proposed DFL is more privacy preserving then the CFL by leveraging transfer learning and reducing the number of training iterations. In addition, we can demonstrate that DFL is also practical in Non-IID settings.

5.3.6. Energy costs

Fig. 9 presents the average energy consumption of proposed PPSS-enabled DFL over 1 round of FL, which includes the local training phase, the model aggregation phase, and the model evaluation phase (i.e., model inference using test data). We evaluate the energy cost of both CFL and DFL as well as the cost of PoFDL consensus (i.e., second-stage aggregation) with and without DP-training. Firstly, when training without DP-SGD, we can see that the CFL is more energy-efficient

Table 5
The evaluation results DFL federated learning in IID.

Provers	Clients	1st round						15th round					
		Without DP			With DP			Without DP			With DP		
		B	W	G	B	W	G	B	W	G	B	W	G
P = 3	K = 20	90.12	86.43	90.12	75.44	73.21	75.44	93.44	92.80	93.99	92.65	91.71	93.40
	K = 40	90.03	89.71	90.03	73.21	73.21	73.21	93.71	93.35	93.85	91.66	90.97	92.26
	K = 80	87.89	85.49	87.89	73.21	73.21	73.21	93.70	93.67	93.90	92.38	90.31	92.09
P = 6	K = 20	92.81	78.92	92.81	78.57	36.75	78.57	93.83	93.55	93,98	89.30	82.76	91.47
	K = 40	91.02	83.12	91.02	73.21	73.21	73.21	93.79	93.42	93.97	93.46	91.30	93.46
	K = 80	90.94	88.13	90.94	73.21	73.21	73.21	93.80	93.67	93.97	93.14	92.01	93.72
P = 8	K = 20	93.21	55.39	93.21	86.98	73.21	86.98	93.86	75.26	93.92	90.92	82.46	92.23
	K = 40	91.30	75.47	91.30	75.18	73.21	75.18	93.84	93.20	94.01	93.00	88.33	93.36
	K = 80	93.25	81.93	93.25	73.21	73.21	73.21	93.84	93.33	93.90	91.98	90.04	92.90

(**B**): Best prover accuracy %; (**W**): Worst prover accuracy %; (**G**): Global model accuracy %;
No-DP : Training without DP-SGD; **DP** : Training with DP-SGD.

Table 6
The evaluation results of DFL federated learning in Non-IID.

Provers	Clients	1st round						15th round					
		No-DP			DP			No-DP			DP		
		B	W	G	B	W	G	B	W	G	B	W	G
P = 3	K = 20	87.46	78.56	87.46	73.21	73.21	73.21	92.41	81.39	92.44	89.55	80.00	91.74
	K = 40	87.78	79.52	87.78	73.21	73.21	73.21	92.32	81.36	92.48	89.15	80.06	89.80
	K = 80	88.87	80.91	88.87	73.21	73.21	73.21	92.45	81.42	92.52	87.89	80.06	88.75
P = 6	K = 20	90.69	73.86	90.69	84.60	71.29	84.60	90.79	83.22	93.28	89.59	78.52	91.83
	K = 40	91.32	78.94	91.32	78.20	73.21	78.20	93.27	81.50	93.75	90.18	80.06	92.27
	K = 80	86.64	80.07	86.64	80.33	73.21	80.33	93.19	81.36	93.55	92.11	80.29	93.18
P = 8	K = 20	92.61	73.65	92.61	73.21	73.21	73.21	91.92	73.90	93.74	90.68	80.16	92.45
	K = 40	91.06	81.25	91.06	75.03	73.21	75.03	93.35	81.51	93.69	90.94	80.05	92.38
	K = 80	92.58	78.46	92.58	73.41	73.21	73.41	93.40	81.10	93.69	91.71	80.84	92.63

(**B**): Best prover accuracy %; (**W**): Worst prover accuracy %; (**G**): Global model accuracy %;
No-DP : Training without DP-SGD; **DP** : Training with DP-SGD.

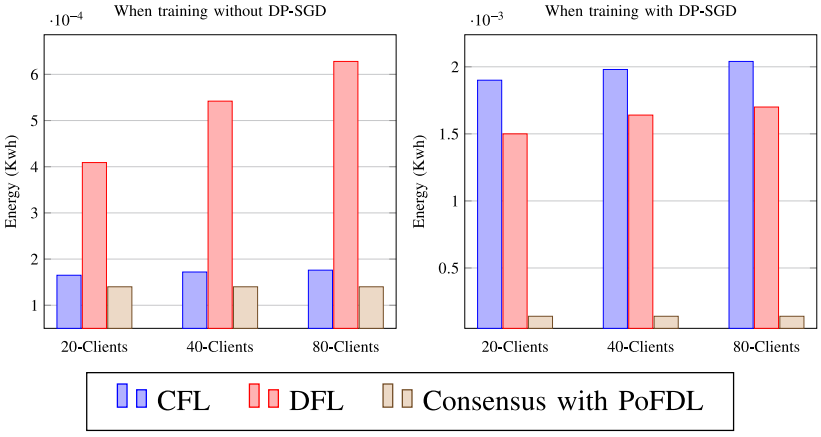


Fig. 9. The average estimation of PPSS-enabled DFL energy consumption over one round of federated learning using Google GPU device(s): Tesla-T4. CFL : Centralized federated Learning; DFL : Decentralized federated Learning.

than the DFL. This is due to the extra operations of model evaluation performed by the DFL which corresponds to the number of participating provers. Secondly, when comparing the energy cost of DP-training, we concluded that training with DP-SGD imposed a considerable cost of energy use. In addition, we concluded that the energy cost of CFL increased by around 10 times when using DP-SGD, while the energy cost of DFL increased by nearly around 3 times. Consequently, we can prove that the DFL is more energy-efficient than the CFL, especially when considering extra rounds of FL and DP-training. This is owing to the fact that the DFL performs additional cost-effective operations of model aggregation

Table 7
Comparison with other state-of-the-art federated learning-based IDS schemes.

Schemes	Year	Dataset	Classification method	Blockchain	Capacity for feature diversity	Machine learning library	Differential Privacy
Al-Marri et al. [8]	2020	NSL-KDD	MLP	No	Low	TensorFlow-Keras	No
Zhao et al. [36]	2020	SEA	RNN-LSTM	No	Low	TensorFlow-Keras	No
Rahman et al. [7]	2020	NSL-KDD	N/A	No	Low	N/A	No
Chen et al. [37]	2020	KDDCup99	GRU	No	Low	PySyft	No
Popoola et al. [6]	2021	Bot-IoT N-BaIoT	DNN	No	Medium Low	TensorFlow-Keras	No
Li et al. [10]	2021	N/A	CNN-GRU	No	N/A	Keras-Flask	No
Kumar et al. [11]	2021	ToN_IoT CICIDS-2017	RNN	Yes	Medium	TensorFlow-Keras	No
Kumar et al. [15]	2021	ToN_IoT Bot-IoT	GB	Yes	Low Medium	Scikit-learn	No
Attota et al. [38]	2021	MQTT	ANN	No	Medium	Pytorch, PySyft	No
Liu et al. [39]	2021	KDDCup99	DNN	Yes	Low	Pytorch-Syft	Yes
Friha et al. [40]	2022	InSDN, MQTTset, CSE-CIC-IDS2018	DNN, CNN, and RNN	No	No	TensorFlow-Keras	No
Tang et al. [41]	2022	CICIDS2017	GRU	No	Medium	PyTorch, PySyft	No
Abdel-Basset et al. [42]	2022	Car-Hacking TON_IoT	AE-DNN	Yes	Low Low	PySyft	No
Li et al. [43]	2022	HDFS BGL	TCN-CNN	No	Low	PyTorch	No
PPSS	/	Edge-IIoT	CNN	Yes	High	Pytorch, Opacus	Yes

Abbreviation & Terms : N/A: Not available, MLP multi layer perceptron, GB: Gradient Boosting, GRU: Gated Recurrent Unit, AE: Auto-Encoder, RNN: Recurrent Neural Network, LSTM, Long short term memory, TCN: Temporal Convolutional Network.

(i.e., transfer learning), which means reducing the number of training rounds compared to the CFL, while the cost of model aggregation and PoFDL consensus is much lower and almost negligible compared to the cost of training.

5.3.7. Blockchain performance and storage overhead

We evaluate the throughput and latency of our blockchain scheme using a qualitative analysis based on how our proposed PoFDL consensus algorithm perform to confirm new blocks. We estimate the latency as the time between when a *Prover* submits a transaction *tx* and when the block containing *tx* is confirmed. Similarly, throughput is defined as the rate at which the *Learning-chain* confirms *tx* in a given amount of time. This includes the computation over a block validation by each *Validator* node and the number of message exchanges required before a block is confirmed. In terms of computing, our PoFDL employs model inference to validate the model-containing blocks, where the cost of inference is substantially lower and relies on the size of the model and the given ValidationSet. In terms of message exchanges, our PoFDL performs $2(N/2+1)$ message rounds, where N corresponds to the number of *Validator* nodes. Secondly, we evaluate the storage overhead of the resultant *Learning-Chain* based on our experiments with the presented task. Table 4 presents the average rate of block data generation within a number of *Prover* nodes. We calculate the storage capacity of each block based on the capacity of its contained data, as shown in Fig. 3, without considering the store overhead of validation, which is heavily depends on the size of used ValidationSet. The trained model used in our experiments is 0.21 MB in size. The average size of a block is 6.46 MB. Although the maximum storage overhead of the *Learning-Chain* for the presented learning task is 50.84 MB, which is predicted to increase with more tasks, we believe that these results are applicable in real-world permissioned blockchain applications that operate at the fog computing layer.

5.4. Comparison between PPSS and other related frameworks

In Table 7, we provide a comparison between our PPSS framework with other state-of-the-art FL-based IDS schemes. The scope of the comparison covers the classification method, used dataset, the use of blockchain, the capacity for feature diversity which denotes the variety of traffic data and gathered features, and finally the use of differential privacy. Tables 8 and 9 provide a comparison between PPSS and other related schemes in both blockchain systems and non-blockchain systems respectively. The scope of the comparison covers the ledger distribution, which refers to the ability to replicate and save an identical copy of the ledger for each participating node; fault tolerance, which refers to the ability to detect failures through distributed consensus protocols; and smart contracts. The PPSS framework uses the proposed PoFDL for consensus and deploys the smart contract to manage transactions and system enforcement. The FL-based IDS is covered by the PPSS framework, which refers to the deployments of privacy-preserving IDS with the ability to identify network cyber-attacks. The encrypted model is adopted by the PPSS framework, which refers to the secure communication of model exchange. The proof of learning is covered by the PPSS framework, which refers to the ability to validate the correctness of submitted models. In addition, the performance evaluation of the PPSS framework is analyzed by data distribution (i.e., IID and Non-IID).

Table 8
Comparison between PPSS and other related federated learning-based schemes in the blockchain systems.

Metric	[11]	[44]	[16]	[45]	[46]	[39]	[47]	[48]	[49]	[42]	[50]	[51]	PPSS
Ledger distribution	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	N	N	Y
Fault tolerance	N	P	N	N	N	N	N	N	N	Y	N	N	Y
Participation in consensus	P	Y	Y	P	Y	Y	Y	Y	Y	Y	Y	Y	Y
Smart contracts	Y	N	N	N	N	N	N	P	P	P	N	N	Y
Federated learning-based IDS	Y	Y	Y	Y	Y	Y	Y	P	P	Y	N	N	Y
Encrypted model	Y	Y	Y	Y	N	Y	Y	Y	Y	Y	N	N	Y
Differential Privacy	N	Y	Y	N	N	Y	N	N	N	N	N	N	Y
Proof of learning	N	P	N	N	N	N	N	N	N	N	Y	Y	Y
IID	Y	P	Y	Y	Y	Y	Y	P	Y	Y	N	Y	Y
Non-IID	N	P	N	N	N	N	N	N	N	N	N	N	Y

Abbreviation & Terms : Y: Yes, N: No, P: Partial.

Table 9
Comparison between PPSS and other related federated learning-based schemes in the non-blockchain systems.

Metric	[6]	[10]	[8]	[7]	[40]	[36]	[43]	PPSS
Ledger distribution	N	N	N	N	N	N	N	Y
Fault tolerance	N	N	N	N	N	N	N	Y
Participation in consensus	N	N	N	N	N	N	N	Y
Smart contracts	N	N	N	N	N	N	N	Y
Federated learning-based IDS	Y	Y	Y	Y	Y	Y	Y	Y
Encrypted model	N	Y	N	N	Y	N	N	Y
Differential Privacy	N	N	N	N	N	N	N	Y
Proof of learning	N	N	N	N	N	N	N	Y
IID	P	P	Y	Y	Y	Y	Y	Y
Non-IID	P	P	N	N	Y	N	Y	Y

Abbreviation & Terms : Y: Yes, N: No, P: Partial.

5.5. Limitations

Experimental results proved the efficiency of our PPSS framework against a heterogeneous dataset and different label distribution-based Non-IID data. However, in cross-silo settings where participants may not share the same feature vector of data because of different device properties and application domains, this would eventually degrade performance and interrupt model sharing because of poor generalization. A prior knowledge of how the data is featured and organized may help overcome these challenges. Furthermore, DP-training has the potential to protect data privacy at the cost of model accuracy based on the amount of noise introduced. Our study demonstrates that leveraging transfer learning and data availability helps reduce the negative effects of DP-training. However, balancing between privacy and model performance is viewed as a design choice and a major limitation of privacy-enhanced FL-based systems.

6. Conclusions and future work

In this paper, we proposed a novel privacy-preserving and secure framework named PPSS to protect industry 4.0/5.0 against privacy breaches and emergent cyber attacks. The PPSS includes a blockchain-enabled FL scheme and a privacy-preserving intrusion detection scheme. In the blockchain-enabled FL scheme, we proposed a differentially private training with an energy-efficient consensus protocol named Proof-of-Federated Deep-Learning (PoFDL) in order to enable privacy-preserving and cross-silo DFL. In the privacy-preserving intrusion detection scheme, we employed the PPSS for anomaly-based IDS using the CNN approach for attack identification. The study analysis and performance evaluations demonstrated the security, reliability, and efficiency of the proposed PPSS framework.

For future work, we plan to further evaluate the proposed framework on open-source blockchain platforms as well as evaluate it with other privacy protection measures such as homomorphic encryption and other unsupervised learning methodologies.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] W.Y.B. Lim, N.C. Luong, D.T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, C. Miao, Federated learning in mobile edge networks: A comprehensive survey, IEEE Commun. Surv. Tutor. 22 (3) (2020) 2031–2063.

[2] D. Hamouda, M.A. Ferrag, N. Benhamida, H. Seridi, Intrusion detection systems for industrial internet of things: A survey, in: 2021 International Conference on Theoretical and Applicative Aspects of Computer Science, ICTAACS, IEEE, 2021, pp. 1–8.

[3] D.C. Nguyen, M. Ding, P.N. Pathirana, A. Seneviratne, J. Li, H.V. Poor, Federated learning for internet of things: A comprehensive survey, IEEE Commun. Surv. Tutor. (2021).

[4] L.U. Khan, W. Saad, Z. Han, E. Hossain, C.S. Hong, Federated learning for internet of things: Recent advances, taxonomy, and open challenges, IEEE Commun. Surv. Tutor. (2021).

[5] D. Li, D. Han, T.-H. Weng, Z. Zheng, H. Li, H. Liu, A. Castiglione, K.-C. Li, Blockchain for federated learning toward secure distributed machine learning systems: a systemic survey, Soft Comput. 26 (9) (2022) 4423–4440.

[6] S.I. Popoola, R. Ande, B. Adebisi, G. Gui, M. Hammoudeh, O. Jogunola, Federated deep learning for zero-day botnet attack detection in IoT edge devices, IEEE Internet Things J. (2021).

[7] S.A. Rahman, H. Tout, C. Talhi, A. Mourad, Internet of things intrusion detection: Centralized, on-device, or federated learning? IEEE Netw. 34 (6) (2020) 310–317.

[8] N.A.A.-A. Al-Marri, B.S. Ciftler, M.M. Abdallah, Federated mimic learning for privacy preserving intrusion detection, in: 2020 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom), IEEE, 2020, pp. 1–6.

[9] K. Li, H. Zhou, Z. Tu, W. Wang, H. Zhang, Distributed network intrusion detection system in satellite-terrestrial integrated networks using federated learning, IEEE Access 8 (2020) 214852–214865.

[10] B. Li, Y. Wu, J. Song, R. Lu, T. Li, L. Zhao, DeepFed: Federated deep learning for intrusion detection in industrial cyber-physical systems, IEEE Trans. Ind. Inform. (2020).

[11] R. Kumar, P. Kumar, R. Tripathi, G.P. Gupta, N. Kumar, M.M. Hassan, A privacy-preserving-based secure framework using blockchain-enabled deep-learning in cooperative intelligent transport system, IEEE Trans. Intell. Transp. Syst. (2021).

[12] M.A. Ferrag, L. Maglaras, DeepCoin: A novel deep learning and blockchain-based energy exchange framework for smart grids, IEEE Trans. Eng. Manage. 67 (4) (2019) 1285–1297.

[13] P. Kumar, R. Kumar, G. Srivastava, G.P. Gupta, R. Tripathi, T.R. Gadekallu, N. Xiong, PPSF: A privacy-preserving and secure framework using blockchain-based machine-learning for IoT-driven smart cities, IEEE Trans. Netw. Sci. Eng. (2021).

[14] P. Kumar, R. Kumar, G.P. Gupta, R. Tripathi, A Distributed framework for detecting DDoS attacks in smart contract-based Blockchain-IoT Systems by leveraging Fog computing, Trans. Emerg. Telecommun. Technol. 32 (6) (2021) e4112.

[15] R. Kumar, P. Kumar, R. Tripathi, G.P. Gupta, T.R. Gadekallu, G. Srivastava, Sp2f: a secured privacy-preserving framework for smart agricultural unmanned aerial vehicles, Comput. Netw. 187 (2021) 107819.

[16] Y. Wan, Y. Qu, L. Gao, Y. Xiang, Privacy-preserving blockchain-enabled federated learning for B5G-Driven edge computing, Comput. Netw. (2021) 108671.

[17] P. Kumar, G.P. Gupta, R. Tripathi, PEFL: Deep privacy-encoding based federated learning framework for smart agriculture, IEEE Micro (2021).

[18] E. Ozfatura, D. Gunduz, H.V. Poor, Collaborative learning over wireless networks: An introductory overview, 2021, arXiv preprint arXiv:2112.05559.

[19] C. Dwork, A. Roth, et al., The algorithmic foundations of differential privacy, Found. Trends Theor. Comput. Sci. 9 (3–4) (2014) 211–407.

[20] H.-N. Dai, Z. Zheng, Y. Zhang, Blockchain for Internet of Things: A survey, IEEE Internet Things J. 6 (5) (2019) 8076–8094.

[21] T. Bocek, B. Stiller, Smart contracts–blockchains in the wings, in: Digital Marketplaces Unleashed, Springer, 2018, pp. 169–184.

[22] R. Cheng, F. Zhang, J. Kos, W. He, N. Hynes, N. Johnson, A. Juels, A. Miller, D. Song, Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts, in: 2019 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2019, pp. 185–200.

[23] M. Castro, B. Liskov, et al., Practical byzantine fault tolerance, in: OSDI, vol. 99, (1999) 1999, pp. 173–186.

[24] F.R. Yu, J. Liu, Y. He, P. Si, Y. Zhang, Virtualization for distributed ledger technology (vDLT), IEEE Access 6 (2018) 25019–25028.

[25] H. Jia, M. Yaghini, C.A. Choquette-Choo, N. Dullerud, A. Thudi, V. Chandrasekaran, N. Papernot, Proof-of-learning: Definitions and practice, in: 2021 IEEE Symposium on Security and Privacy, SP, IEEE, 2021, pp. 1039–1056.

[26] M. Abadi, A. Chu, I. Goodfellow, H.B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, Association for Computing Machinery, New York, NY, USA, 2016, pp. 308–318, http://dx.doi.org/10.1145/2976749.2978318.

[27] S. De Angelis, L. Aniello, R. Baldoni, F. Lombardi, A. Margheri, V. Sassone, PBFT vs proof-of-authority: Applying the CAP theorem to permissioned blockchain, 2018.

[28] R. Shokri, M. Stronati, C. Song, V. Shmatikov, Membership inference attacks against machine learning models, in: 2017 IEEE Symposium on Security and Privacy, SP, IEEE, 2017, pp. 3–18.

[29] M. Fredrikson, S. Jha, T. Ristenpart, Model inversion attacks that exploit confidence information and basic countermeasures, in: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, 2015, pp. 1322–1333.

[30] Y. Liu, Y. Lan, B. Li, C. Miao, Z. Tian, Proof of Learning (PoLe): Empowering neural network training with consensus building on blockchains, Comput. Netw. 201 (2021) 108594.

[31] M.A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning, IEEE Access 10 (2022) 40281–40306.

[32] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (7553) (2015) 436–444.

[33] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, et al., Opacus: User-friendly differential privacy library in PyTorch, 2021, arXiv preprint arXiv:2109.12298.

[34] X. Qiu, T. Parcollet, J. Fernandez-Marques, P.P.B. de Gusmao, D.J. Beutel, T. Topal, A. Mathur, N.D. Lane, A first look into the carbon footprint of federated learning, 2021, arXiv preprint arXiv:2102.07627.

[35] L.F.W. Anthony, B. Kanding, R. Selvan, Carbontracker: Tracking and predicting the carbon footprint of training deep learning models, 2020, arXiv preprint arXiv:2007.03051.

[36] R. Zhao, Y. Yin, Y. Shi, Z. Xue, Intelligent intrusion detection based on federated learning aided long short-term memory, Phys. Commun. 42 (2020) 101157.

[37] Z. Chen, N. Lv, P. Liu, Y. Fang, K. Chen, W. Pan, Intrusion detection for wireless edge networks based on federated learning, IEEE Access 8 (2020) 217463–217472.

[38] D.C. Attota, V. Mothukuri, R.M. Parizi, S. Pouriyeh, An ensemble multi-view federated learning intrusion detection for iot, IEEE Access 9 (2021) 117734–117745.

[39] H. Liu, S. Zhang, P. Zhang, X. Zhou, X. Shao, G. Pu, Y. Zhang, Blockchain and federated learning for collaborative intrusion detection in vehicular edge computing, IEEE Trans. Veh. Technol. 70 (6) (2021) 6073–6084.

[40] O. Friha, M.A. Ferrag, L. Shu, L. Maglaras, K.-K.R. Choo, M. Nafaa, FELIDS: Federated learning-based intrusion detection system for agricultural Internet of Things, J. Parallel Distrib. Comput. 165 (2022) 17–31.
[41] Z. Tang, H. Hu, C. Xu, A federated learning method for network intrusion detection, Concurr. Comput.: Pract. Exper. (2022) e6812.
[42] M. Abdel-Basset, N. Moustafa, H. Hawash, I. Razzak, K.M. Sallam, O.M. Elkomy, Federated intrusion detection in blockchain-based smart transportation systems, IEEE Trans. Intell. Transp. Syst. (2022).
[43] B. Li, S. Ma, R. Deng, K.-K.R. Choo, J. Yang, Federated anomaly detection on system logs for the internet of things: A customizable and communication-efficient approach, IEEE Trans. Netw. Serv. Manag. (2022).
[44] A. Islam, A. Al Amin, S.Y. Shin, FBI: A federated learning-based blockchain-embedded data accumulation scheme using drones for internet of things, IEEE Wirel. Commun. Lett. 11 (5) (2022) 972–976.
[45] J. Wei, Q. Zhu, Q. Li, L. Nie, Z. Shen, K.-K.R. Choo, K. Yu, A redactable blockchain framework for secure federated learning in industrial internet-of-things, IEEE Internet Things J. (2022).
[46] S. Singh, S. Rathore, O. Alfarraj, A. Tolba, B. Yoon, A framework for privacy-preservation of IoT healthcare data using Federated Learning and blockchain technology, Future Gener. Comput. Syst. 129 (2022) 380–388.
[47] A. Lakhan, M.A. Mohammed, J. Nedoma, R. Martinek, P. Tiwari, A. Vidyarthi, A. Alkhayyat, W. Wang, Federated-learning based privacy preservation and fraud-enabled blockchain IoMT system for healthcare, IEEE J. Biomed. Health Inf. (2022).
[48] Y. Liu, W. Yu, Z. Ai, G. Xu, L. Zhao, Z. Tian, A blockchain-empowered federated learning in healthcare-based cyber physical systems, IEEE Trans. Netw. Sci. Eng. (2022).
[49] Z. Peng, J. Xu, X. Chu, S. Gao, Y. Yao, R. Gu, Y. Tang, VFChain: enabling verifiable and auditable federated learning via blockchain systems, IEEE Trans. Netw. Sci. Eng. 9 (1) (2021) 173–186.
[50] F. Bravo-Marquez, S. Reeves, M. Ugarte, Proof-of-learning: a blockchain consensus mechanism based on machine learning competitions, in: 2019 IEEE International Conference on Decentralized Applications and Infrastructures, DAPPCON, IEEE, 2019, pp. 119–124.
[51] C. Qiu, X. Wang, H. Yao, J. Du, F.R. Yu, S. Guo, Networking integrated cloud–edge–end in IoT: A blockchain-assisted collective Q-learning approach, IEEE Internet Things J. 8 (16) (2020) 12694–12704.